

Solidity

Hardhat 3

Sepolia

● ĐỒ ÁN MÔN CHUỖI KHỐI & ỨNG DỤNG

Hệ thống quản lý thuê nhà bằng **smart contract**

Tự động hoá đặt cọc, thanh toán, bàn giao và tất toán hợp đồng thuê nhà trên Ethereum — tiền cọc do contract giữ khách quan, mọi giao dịch minh bạch, không cần bên trung gian con người.

THỰC HIỆN

[Tên bạn] · [MSSV]

MẠNG TRIỂN KHAI

Ethereum Sepolia Testnet

TEST TỰ ĐỘNG

32/32 pass

Thuê nhà truyền thống có 3 điểm yếu

Quy trình dựa trên giấy tờ, tiền mặt/chuyển khoản và lòng tin giữa hai bên — không có bên thứ ba khách quan đứng ra đảm bảo.

1

Tiền cọc thiếu minh bạch

Chủ nhà tự giữ tiền cọc. Người thuê chỉ có thể "tin tưởng" chủ nhà sẽ hoàn đúng hạn, đúng số tiền khi kết thúc hợp đồng.

2

Không có bằng chứng khách quan

"Đã trả tiền kỳ này chưa", "đã bàn giao chưa" — chỉ dựa vào lời nói/tin nhắn, khó tra cứu lại khi có tranh chấp xảy ra sau này.

3

Xử lý thủ công, tốn thời gian

Chủ nhà tự theo dõi từng người thuê, từng kỳ thanh toán, tự tính khấu trừ cọc — dễ sai sót, không có luật cố định áp dụng nhất quán.

Giải pháp: đưa nghiệp vụ lỗi lên smart contract

Một đoạn mã đã triển khai, không ai sửa được, đóng vai "bên trung gian không thể thiên vị" thay cho con người.



Giữ tiền cọc khách quan

Tiền cọc do **RentaLManager** giữ (không vào ví chủ nhà ngay) — chỉ giải ngân theo đúng luật đã lập trình sẵn.



Ghi nhận giao dịch không thể chỉnh sửa

Mọi lần đặt cọc, trả tiền, bàn giao, tắt toán đều là *event* trên blockchain — tra cứu lại vĩnh viễn, không ai xoá/sửa.



Tự động hoá luật hợp đồng

Chủ nhà không tự thuê được nhà mình, khấu trừ không vượt quá cọc, chỉ đúng người mới trả tiền được — contract từ chối ngay giao dịch sai.



Bằng chứng sở hữu hợp đồng bằng token

Mỗi lượt thuê thành công = 1 token **ERC-721** không chuyển nhượng, mint thẳng cho người thuê ngay khi đặt cọc.

4 đối tượng người dùng (actor)

Quyền hạn xác định bằng dữ liệu on-chain (`msg.sender`), không phải bằng vai trò được admin duyệt trước — trừ đúng 1 ngoại lệ: trọng tài.



Không cấp phép

Chủ nhà (Landlord)

Đăng tài sản, hủy tin nhầm, nhận tiền thuê định kỳ, đề xuất khấu trừ cọc khi kết thúc.



Không cấp phép

Người thuê (Tenant)

Đặt cọc kích hoạt hợp đồng, trả tiền định kỳ, xác nhận bàn giao, khiếu nại nếu không đồng ý khấu trừ.



ARBITER_ROLE

Trọng tài (Arbiter)

Được cấp quyền qua `grantRole` — bỏ phiếu xử lý tranh chấp, chỉ hiệu lực khi đủ số phiếu đồng thuận (multisig).



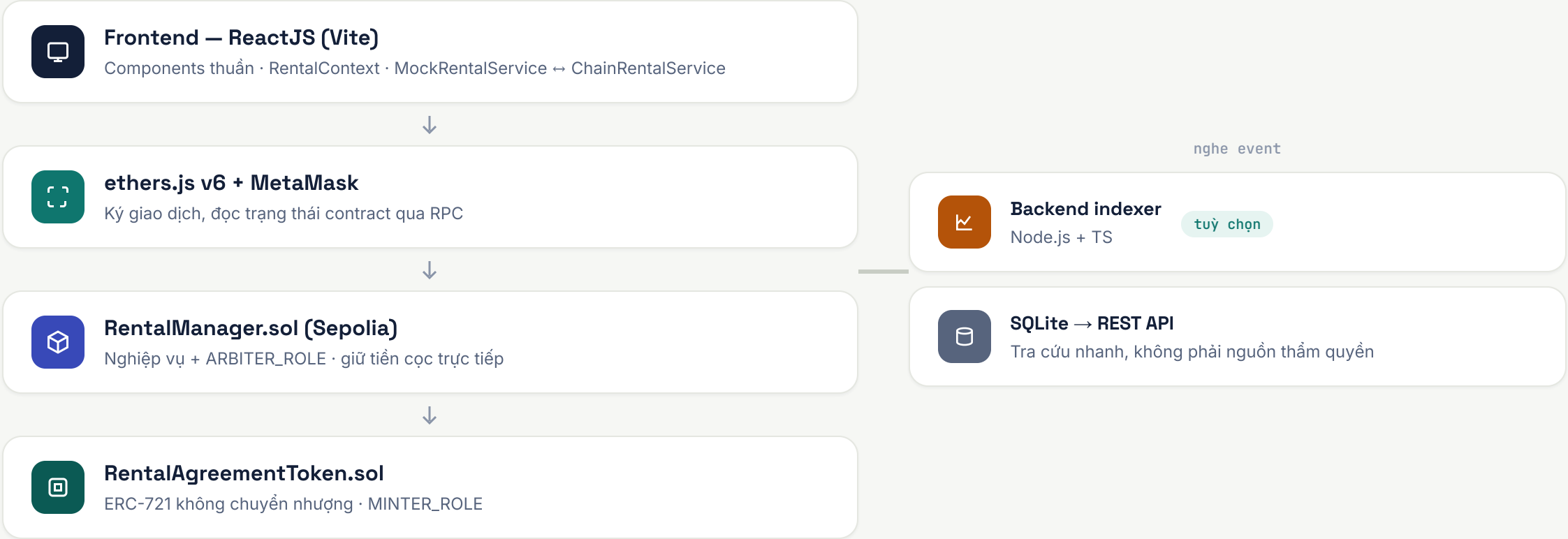
Chỉ đọc

Người xem công khai

Bất kỳ ai cũng đọc được toàn bộ lịch sử giao dịch (minh bạch), nhưng không có quyền ghi bất cứ gì.

Tách rõ giao diện — logic đọc/ghi — smart contract

Component UI không biết mình đang chạy dữ liệu mẫu hay blockchain thật — nhờ lớp RentalService làm interface chung.



Vì sao chọn ERC-721 làm token đại diện hợp đồng

Mỗi lượt thuê là **duy nhất** (chủ nhà, tiền thuê, tiền cọc khác nhau) — không thể "thay thế lẫn nhau" như ERC-20.

TIÊU CHUẨN	PHÙ HỢP?	LÝ DO
ERC-20	Không chọn	Token thay thế lẫn nhau — không biểu diễn được "hợp đồng thuê cụ thể nào".
ERC-721	✓ Chọn	Mỗi tokenId là duy nhất, tra cứu "ai đang thuê" bằng ownerOf().
ERC-1155	Không chọn	Dùng khi cần nhiều loại tài sản/phát hành theo lô — không cần thiết, chỉ 1 loại tài sản.

KHÔNG CHUYỂN NHƯỢNG

Khoá `transferFrom` / `approve` / `burn` — chủ nhà chỉ đồng ý cho đúng 1 người thuê, không cho "bán lại" quyền thuê.

TOKENID == PROPERTYID

Mỗi property chỉ thuê đúng 1 lần trong vòng đời hiện tại → không gian id không bao giờ trùng, frontend không cần biết khái niệm tokenId riêng.

2 CONTRACT TÁCH BIỆT

`RentalAgreementToken` (token, `MINTER_ROLE`) tách khỏi `RentalManager` (nghịệp vụ) — đúng mô hình token/manager.

8/8 chức năng theo đúng đề bài

Đặt cọc = tạo hợp đồng luôn (không tách bước riêng) — kích hoạt trạng thái Active và mint token trong cùng 1 giao dịch.

1 Đăng thông tin tài sản
`listProperty()` ✓

2 Đặt cọc → tạo hợp đồng
`rentProperty()` ✓

3 Thanh toán định kỳ
`payRent()` ✓

4 Xác nhận bàn giao
`confirmHandover()` ✓

5 Hoàn/khấu trừ tiền cọc
`proposeSettlement` → `acceptSettlement` ✓

6 Kết thúc hợp đồng
`_settle()` (nội bộ) ✓

7 Theo dõi lịch sử thanh toán
Đọc trực tiếp event blockchain ✓

+ Bổ sung ngoài yêu cầu
`cancelListing()` — huỷ tin đăng nhậm ✓

Cơ chế trọng tài khi có tranh chấp

Thay endLease cũ (chủ nhà quyết định & thực thi ngay) bằng luồng nhiều bước, có đường thoát cho cả 2 phía.

proposeSettlement

Chủ nhà đề xuất mức khấu trừ



Người thuê phản hồi

Đồng ý

Người thuê chấp nhận mức đề xuất → tất toán ngay lập tức, không cần bên thứ 3.

- `acceptSettlement()` → `_settle()`

Không đồng ý → Khiếu nại

Chuyển trạng thái Disputed — không bên nào đơn phương quyết định được nữa.

- `disputeSettlement()`
- `voteOnDispute()` — chỉ `ARBITER_ROLE`

☆ ≥2 trọng tài đồng thuận cùng 1 mức → tự tất toán

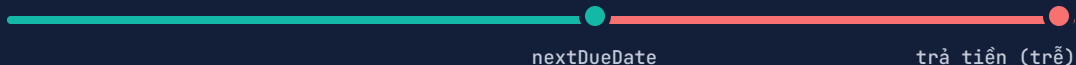
Phạt thanh toán trễ

Trả sau `nextDueDate` thì bắt buộc cộng thêm phạt — không có cách "lách", giao dịch thiếu tiền phạt bị từ chối thẳng.

CÔNG THỨC TÍNH (PAYRENT)

```
penalty = block.timestamp > nextDueDate
? (monthlyRent * lateFeeBps) / 10000
: 0
```

```
require(msg.value == monthlyRent + penalty)
```



⚙️ Mặc định 5%

`LateFeeBps = 500` (basis points), cấu hình được lúc deploy qua tham số Hardhat Ignition.

📅 Hạn kỳ tiếp theo

Tính từ **hạn cũ**, không tính từ lúc trả trễ — `nextDueDate += rentPeriod`, tránh "hoãn dây chuyền".

✅ Đã kiểm thử

Dùng `networkHelpers.time.increase()` mô phỏng quá hạn — xác nhận bắt buộc phạt, chặn trả thiếu.

Multisig cho giải ngân tiền đặt cọc

Không dùng TimelockController riêng như đề xuất ban đầu — đếm phiếu trực tiếp on-chain, đơn giản hơn, không cần contract phụ trợ.

A1

Trọng tài #1 vote 0.5 ETH

`disputeVotes[id][0.5] = 1`

A2

Trọng tài #2 vote 0.5 ETH

`disputeVotes[id][0.5] = 2 ≥ ngưỡng`

A3

Trọng tài #3 vote 0.8 ETH (mức khác)

Không cộng dồn — khác mức, không tính



KHI ĐỦ `ARBITERAPPROVALSREQUIRED`

Tự động tắt toán ngay

2 trọng tài **độc lập** cùng đồng thuận đúng 1 mức khấu trừ → gọi

`_settle()` : hoàn phần còn lại cho người thuê, chuyển phần khấu trừ cho chủ nhà.

Chặn 1 trọng tài đơn lẻ tự quyết (`onlyRole`), chặn vote 2 lần cùng 1 tranh chấp (`hasVotedOnDispute`).

cancelListing — huỷ tin đăng nhầm

Phát sinh từ chính quá trình dùng thử thật: dữ liệu blockchain **không sửa lại được** sau khi ghi.

⚠ Vấn đề thực tế

Các trường của `Property` (giá, cọc, mô tả...) chỉ ghi được **một lần** trong `listProperty`. Đăng nhầm giá → tin nằm sai **vĩnh viễn**.

Chủ nhà cũng **không tự thuê được** tin của chính mình để "dọn" nó đi — `rentProperty` chặn `msg.sender = p.landlord`.

✅ Giải pháp

`cancelListing(id)` — chỉ chủ nhà, chỉ khi còn `Listed` (chưa ai đặt cọc), chuyển sang trạng thái `Cancelled` mới.

- ✓ Chủ nhà huỷ được tin còn trống
- ✓ Người khác không huỷ được
- ✓ Không huỷ được tin đã có người thuê

Kết quả kiểm thử tự động

Hardhat Test + Chai (Mocha) — bao phủ nghiệp vụ, token, bảo mật, và cả tính năng bổ sung.

32/32

test case pass

`npx hardhat test`

Nghiệp vụ cơ bản

9

Đăng tin, đặt cọc đúng/sai tiền, chặn tự thuê, tiền cọc giữ ở contract, trả tiền đúng chủ, CID ảnh, ghi chú.

Hủy tin đăng

4

Chủ nhà hủy được, người khác không hủy được, không hủy tin đã thuê, tin hủy không thuê lại được.

Token đại diện hợp đồng

5

Mint đúng người, chặn cả 2 overload `safeTransferFrom`, chặn mint trái phép, token tồn tại sau kết thúc.

Phạt trả trễ

2

Time-travel mô phỏng quá hạn, xác nhận cộng phạt, hạn kỳ tiếp tính từ hạn cũ.

Trọng tài / multisig

11

Đề xuất/đồng ý/khiếu nại, chặn vote trái phép, 1 phiếu chưa đủ, 2 phiếu cùng mức tự tất toán, chặn vote 2 lần.

Bảo mật

1

Contract độc hại cố gọi lại `rentProperty` trong `onERC721Received` — `nonReentrant` chặn được.

Rà soát bảo mật — kể cả 1 lỗi thật tìm thấy khi test

Slither không chạy tự động được (chưa tương thích Hardhat 3) → rà soát thủ công theo đúng nhóm lỗi Slither hay phát hiện.

12

hạng mục rà soát

0

lỗi mất/khoá tiền

3

Confirmed – rủi ro cần xử lý

1

xảy ra thật trong lúc test

Confirmed • phát hiện thật, không phải lý thuyết

Xung đột lợi ích khi bỏ phiếu trọng tài

PHÁT HIỆN

Ví admin (đồng thời đóng vai chủ nhà lúc test) **tự bỏ phiếu được** cho tranh chấp của chính mình — vì `ARBITER_ROLE` được cấp mặc định cho admin trong constructor.

NGUYÊN NHÂN

`voteOnDispute()` chỉ kiểm tra `ARBITER_ROLE`, không kiểm tra người vote có phải chính chủ nhà/người thuê của tranh chấp đó không.

ĐÃ XỬ LÝ

Thu hồi `ARBITER_ROLE` khỏi ví admin, cấp cho ví độc lập (`revoke-arbiter.ts`). Chưa chặn ở mức contract — ghi nhận trung thực là giới hạn còn lại.

Contract trên Sepolia • Giao diện trên Vercel

Không chỉ chạy local — đã lên mạng testnet thật, ai cũng truy cập được từ trình duyệt, không cần cài gì.



Smart contract

Ethereum Sepolia Testnet

- ✓ Deploy bằng **Hardhat Ignition**, bí mật lưu trong **Hardhat keystore** (không lộ private key)
- ✓ Kiểm thử thật bằng nhiều ví MetaMask đóng các vai khác nhau
- ✓ Xác minh độc lập được trên Sepolia Etherscan — dữ liệu thật, không giả lập



Giao diện web

React (Vite) trên Vercel

- ✓ Tự động build & deploy lại mỗi khi push code lên GitHub
- ✓ Kết nối MetaMask thật, không dùng dữ liệu mẫu (**VITE_USE MOCK=false**)
- ✓ Ảnh phòng lưu trên IPFS (Pinata) — chỉ tham chiếu CID on-chain

Ghi nhận trung thực — không tick giả

Những gì chưa làm được cho production thật, nêu rõ thay vì bỏ qua.

Không time lock cho quyết định trọng tài

Đủ phiếu là tất toán ngay — không có độ trễ để bên còn lại phản ứng nếu trọng tài thông đồng.

Xung đột lợi ích chưa chặn ở contract

Mới giảm thiểu bằng quản trị vai trò (thu hồi/cấp lại), `voteOnDispute` chưa tự kiểm tra người vote có phải chính chủ nhà/người thuê.

Admin tập trung quyền lực

Chỉ 1 ví `DEFAULT_ADMIN_ROLE` cấp được `ARBITER_ROLE` — nên chuyển sang ví multisig thật ở production.

Chưa có Pausable

Không có cách tạm dừng hệ thống nếu phát hiện lỗi nghiêm trọng sau khi deploy.

Slither chưa chạy tự động được

`cryptic-compile` chưa hỗ trợ định dạng Hardhat 3 — đã rà soát thủ công thay thế.

SQLite, chưa audit độc lập

Phù hợp demo/đồ án; production thật cần PostgreSQL, Foundry fuzz test, và audit từ bên thứ 3.

12/12 chức năng hoàn thành, deploy thật, kiểm thử đầy đủ — sẵn sàng bảo vệ đồ án.

8/8

chức năng tối thiểu

3/3

chức năng nâng cao

+1

bổ sung: cancelListing

32/32

test tự động

Sepolia

+ Vercel — chạy thật

Cảm ơn thầy/cô và các bạn đã lắng nghe.

Q&A